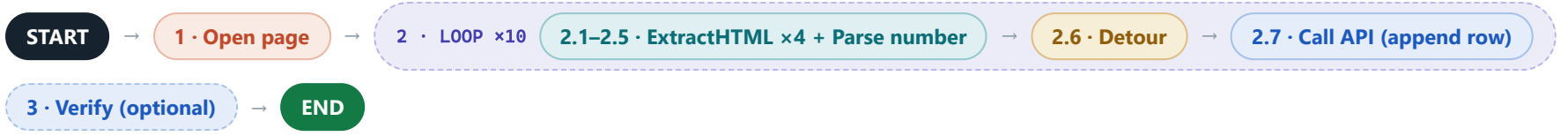
TENNR · IMPLEMENTATION ENGINEER · TAKE-HOME ASSIGNMENT

Wikipedia → Google Sheets

An RPA workflow that extracts **Year, Winner, Score, Runners-up** from the first 10 rows of the *"List of FIFA World Cup finals"* table and appends each row to a Google Sheet via the Google Sheets API (configured exactly as a Postman request) — built **exclusively from the 9 steps in the provided IE Step Library**.

Candidate · Bhunesh Bansal **Date** · 8 July 2026 **Source** · en.wikipedia.org/wiki/List_of_FIFA_World_Cup_finals **Target** · Google Sheet — tab `Sheet1`, columns A:D



A Reading guide — the Step Library

Constraint honored: every executable node is one of the library's 9 steps — 6 are used, the other 3 are listed with why they're unnecessary. The brief says append to an "Excel sheet"; the Google Sheets API only writes to a *native* Google Sheet, so the target is a converted Sheet (setup S1).

Open page
starts the RPA browsing session
`used ×1`

Loop
fixed-count iteration, index from 1
`used ×1 (10 iterations)`

ExtractHTML
DOM path in → element text out
`used ×4 per iteration`

Parse number
numeric string → number + row-validity gate
`used ×1 per iteration`

Detour
"if" — run branch when true, then resume
`used ×1 per iteration`

Call API
configured exactly like Postman
`used ×1 per iteration (+1 optional QA)`

Fill text field
types into an input
`not needed – read-only page, nothing to type`

Click
simulated mouse click
`deliberately avoided – clicking a column header would re-sort rows and scramble tr[index]`

Check Box
toggles a checkbox
`not needed – page has no checkboxes`

Step-library conformance — every node is a library step, unchanged

Each row is quoted verbatim from the Step Guide's *Required Input / Output* columns, next to how this workflow uses it. Nothing invented, nothing outside the 9 steps.

LIBRARY STEP	REQUIRED INPUT (GUIDE)	OUTPUT (GUIDE)	USED IN	THIS WORKFLOW: INPUT → OUTPUT
Open page	Page URL	— (none)	Node 1	the Wikipedia URL → live browsing session
Loop	Number of iterations *or* list	same (index from 1)	Node 2	`10` → index `i` = 1…10
ExtractHTML	Element DOM path	Text	Nodes 2.1, 2.3, 2.4, 2.5	XPath (with `{{index}}` substituted) → cell text
Parse number	String	**String** ⚠ desc. says "Int"	Node 2.2	`year_text` → `year` — row-validity gate; type caveat in **#11**
Detour	— (blank in guide)	— (blank)	Node 2.6	"all three text fields non-empty?" → run append branch, then resume
Call API	— (blank; "same as a call from Postman")		Nodes 2.7, 3	POST `…values/Sheet1!A:D:append` → HTTP 200 (full config in panel D)
Fill text field · Click · Check Box — not used; each is listed in the legend above with the reason it's unnecessary for a read-only extraction.				

Two honest notes: (1) **Parse number**'s guide row contradicts itself — Output column says "String", description says "Int"; the workflow doesn't depend on the resolution (edge case #11). (2) One assumption, and it's the intended one: the Loop's `index` is interpolated into ExtractHTML's DOM-path string — the reason the Loop step and the "index starts from 1" note exist.

B One-time setup (before the workflow runs)

Preconditions only — these are **not** workflow steps and use nothing from the Step Library.

PREPARED ONCE, REUSED ON EVERY RUN

S1 Target spreadsheet

Create a Google Sheet (or convert the uploaded Excel file to Google Sheets format — the API can only write to native Sheets). Tab `Sheet1`; headers `Year | Winner | Score | Runners-up` in A1:D1. Copy the **spreadsheetId** from the URL between `/d/` and `/edit`.

S2 Google Cloud

Create a project → enable the **Google Sheets API** → create OAuth 2.0 client credentials (the sheet's owner account must grant access).

S3 Postman auth

OAuth 2.0 (Authorization Code) → obtain `{{ACCESS_TOKEN}}` with scope `…/auth/spreadsheets`. Full config in panel D.

S4 Capture DOM paths

Per the resource video: DevTools → Inspect element → right-click the cell node → **Copy** → **Copy XPath** (or **Copy selector** for CSS). Parameterise the row by the loop index. Both forms live-verified in panel E.

C The workflow — flowchart

One straight spine: open the page, loop over the 10 finals rows, and inside every iteration extract 4 fields, type-check the year, guard with a Detour, and append the row via one API call. Loop index `i` does triple duty: iteration counter ↔ table row `tbody/tr[i+1]` (row 1 of tbody is the header) ↔ sheet row `i+1`.

START

1 Open the source page — Open page

Begins the RPA browsing session (per library note) and loads the article that hosts the target table.

INPUT `https://en.wikipedia.org/wiki/List_of_FIFA_World_Cup_finals`

Page URL — the only required input.

OUTPUT Live browser session with the page fully rendered. The table keeps its header row as the first **<tbody>** row — verified in a live browser on 7 Jul 2026 (today's page no longer relocates it into `<thead>` the way the May-2024 resource video shows) — so data rows are **tbody/tr[2] … tbody/tr[23]**, and the 1930 final is **tr[2]**.

Keep default (chronological) sort. No clicks on column headers — sorting re-orders `tr` indices and would corrupt the extraction. The table renders on page load; no interaction is needed.

page ready

2 Loop — once per finals row — Loop

Input: number of iterations = **10** · **Output:** index `i` = 1, 2, … 10 (library: index starts from 1)

- i = 1 → tr[2] = 1930 … i = 10 → tr[11] = 1974
- error inside an iteration ⇒ run continues at next index (library note)
- rows are processed top-to-bottom, so sheet order = table order
- row selector tr[{{index}}+1] stays valid XPath after substitution — tr[1+1] ≡ tr[2] (verified live)
- how: the Loop substitutes {{index}} into each DOM-path string, then ExtractHTML resolves it

2.1 Extract "Year" — ExtractHTML

INPUT
```
//*[@id="mw-content-text"]/div[2]/section[2]/table[3]/tbody/tr[{{index}}+1]/th/a
```

Element DOM path (XPath) — year link inside the row-header cell. `+1` skips the header row.

OUTPUT Text → `year_text` e.g. i=1 → **"1930"**

2.2 Convert year to a number — Parse number

INPUT `year_text`

String, e.g. **"1930"**

OUTPUT `year`

Numeric, e.g. **1930** — appended unquoted so column A holds a real number, not text.

Primary purpose is a row-guard, not the conversion: if markup ever shifts and a non-numeric string reaches this step, it errors → the Loop skips to the next index (library behaviour), so a malformed row can never reach the sheet. **Library type note:** the step's *description* says it returns an Int while its *Output* column lists "String" — the guard holds either way (see edge case #11).

next index · i ≤ 10 · repeat ↺

2.3 Extract “Winner” — ExtractHTML

INPUT

```
//*[@id="mw-content-text"]/div[2]/section[2]/table[3]/tbody/tr[{{index}}+1]/td[1]
```

Whole Winners cell. Deliberately **not** the country link: its own path varies row to row (`td[1]/a` on 1930, `td[1]/span/a` on 1954 — verified live), so the cell is the only tail that works for all 10 rows.

OUTPUT Text → `winner` e.g. i=1 → **"Uruguay"** (trimmed — see edge case #9)

↓

2.4 Extract “Score” — ExtractHTML

INPUT

```
//*[@id="mw-content-text"]/div[2]/section[2]/table[3]/tbody/tr[{{index}}+1]/td[2]
```

Whole cell — keeps the “(a.e.t.)” qualifier where a final went to extra time (1934, 1966).

OUTPUT Text → `score` e.g. i=1 → **"4–2"**, i=2 → **"2–1 (a.e.t.)"**

Scores use an **en dash (–, U+2013)**, not a hyphen. Row 4 (1950) carries a footnote marker in the same cell; the library has no string-editing step to strip it, so the whole cell is the honest choice under the constraint — see edge case **#3**.

↓

2.5 Extract “Runners-up” — ExtractHTML

INPUT

```
//*[@id="mw-content-text"]/div[2]/section[2]/table[3]/tbody/tr[{{index}}+1]/td[3]
```

Whole Runners-up cell (country link is span-wrapped on every row — cell-level is uniform).

OUTPUT Text → `runners_up` e.g. i=1 → **"Argentina"** (trimmed)

↓

2.6 ◇ Append guard — is the row complete? — Detour

Library: “conditional execution like an ‘if’ statement; if true, run branch then resume the run.” One branch, one condition.

INPUT

```
winner ≠ "" AND score ≠ "" AND runners_up ≠ ""
```

`year` is already validated by step 2.2 — this guards the three text fields against a silent partial extraction.

OUTPUT true → branch executes (2.7), then the run resumes · false → branch skipped, loop moves on

↓

condition FALSE

Branch does not run. Nothing is appended — **no partial rows ever reach the sheet**.

Run resumes → next index.

condition TRUE → run branch

2.7 Append this row to Google Sheets — Call API

One POST per row, configured exactly as a Postman request — full spec in panel D.

INPUT

```
POST …/values/Sheet1!A:D:append
```

Body → `"values": [[ {{year}}, "{{winner}}", "{{score}}", "{{runners_up}}" ]]`

e.g. i=1 → **[[1930, "Uruguay", "4–2", "Argentina"]]**

OUTPUT HTTP 200 · `updates.updatedRange = "Sheet1!A{{index+1}}:D{{index+1}}"` (i=1 → A2:D2)

If the call fails (401/429/network), the iteration errors and the Loop **skips to the next index** — one bad row never blocks the other nine.

both paths converge → run resumes → Loop advances to next index

after iteration 10 → exit loop ▼

all rows processed

3 Verify the write (optional QA — recommended) — Call API

INPUT

```
GET https://sheets.googleapis.com/v4/spreadsheets/{{SPREADSHEET_ID}}/values/Sheet1!A:D
```

OUTPUT 200 with `values` array of **11 rows** (1 header + 10 data) — confirms every append landed.

↓

END

Final state: Sheet1!A2:D11 = the first 10 FIFA World Cup finals, 1930 → 1974 (full data in panel F)

D The API call — Google Sheets `values:append` via Postman

Exact configuration for step 2.7. Everything in `{{double_braces}}` is a Postman/workflow variable.

POST `https://sheets.googleapis.com/v4/spreadsheets/{{SPREADSHEET_ID}}/values/{{RANGE}}:append`

PATH VARIABLES

VARIABLE	VALUE / WHERE IT COMES FROM
`SPREADSHEET_ID`	From the sheet URL: `docs.google.com/spreadsheets/d/<this>/edit`
`RANGE`	`Sheet1!A:D` — the table to append to (Postman URL-encodes `!` and `:` automatically)

QUERY PARAMETERS

PARAM	WHY
`valueInputOption=RAW`	Write values literally. `USER_ENTERED` would let Sheets re-interpret strings (dash-separated scores can coerce to dates); RAW is deterministic.
`insertDataOption=INSERT_ROWS`	Always add new rows below the existing table — never overwrite.
`includeValuesInResponse=true`	Optional: response echoes what was written — instant per-row verification.

HEADERS

`Content-Type`	`application/json`
`Authorization`	`Bearer {{ACCESS_TOKEN}}` (set automatically by Postman's OAuth 2.0 helper)

AUTH — POSTMAN OAUTH 2.0 (AUTHORIZATION CODE)

Auth URL	`https://accounts.google.com/o/oauth2/v2/auth`
Token URL	`https://oauth2.googleapis.com/token`
Scope	`https://www.googleapis.com/auth/spreadsheets`
Callback	`https://oauth.pstmn.io/v1/callback`

REQUEST BODY — TEMPLATE (ITERATION I)

```
{
  "range": "Sheet1!A:D",
  "majorDimension": "ROWS",
  "values": [
    [ {{year}}, "{{winner}}", "{{score}}", "{{runners_up}}" ]
  ]
}
```

CONCRETE EXAMPLE — ITERATION 1

```
{
  "range": "Sheet1!A:D",
  "majorDimension": "ROWS",
  "values": [
    [ 1930, "Uruguay", "4–2", "Argentina" ]
  ]
}
```

SUCCESS RESPONSE — 200 OK (ITERATION 1)

```
{
  "spreadsheetId": "{{SPREADSHEET_ID}}",
  "tableRange": "Sheet1!A1:D1",
  "updates": {
    "updatedRange": "Sheet1!A2:D2",
    "updatedRows": 1,
    "updatedColumns": 4,
    "updatedCells": 4
  }
}
```

FAILURE MODES

`401`	Token expired → refresh in Postman (Get New Access Token).
`403`	Sheets API not enabled, or account lacks edit access to the sheet.
`404`	Wrong `SPREADSHEET_ID`.
`429`	Quota (60 write req/min/user). 10 sequential appends sit far below it.

Inside the Loop, any failure only skips that row (library note) — remaining iterations still append. `:append` is **not idempotent**: re-running the workflow duplicates rows, so clear A2:D11 before a re-run.

E DOM paths & CSS selectors — captured & verified

The resource video (which demonstrates on this very table) shows the DevTools context menu — right-click the **cell** (th/td) → **Copy** → it offers both **Copy XPath** and **Copy selector** (CSS). Both forms are given below, so ExtractHTML can take whichever the RPA tool expects. Each was evaluated against all 10 rows in a live Chrome session: **XPath 40/40 and CSS 40/40 — 80/80 checks passed.**

XPATH — DEVTOOLS "COPY XPATH"

FIELD	XPATH TEMPLATE (ELEMENT DOM PATH)	OUTPUT @ I = 1
Year	`//*[@id="mw-content-text"]/div[2]/section[2]/table[3]/tbody/tr[{{index}}+1]/th/a`	`"1930"`
Winner	`//*[@id="mw-content-text"]/div[2]/section[2]/table[3]/tbody/tr[{{index}}+1]/td[1]`	`"Uruguay "` → trim
Score	`//*[@id="mw-content-text"]/div[2]/section[2]/table[3]/tbody/tr[{{index}}+1]/td[2]`	`"4–2"`
Runners-up	`//*[@id="mw-content-text"]/div[2]/section[2]/table[3]/tbody/tr[{{index}}+1]/td[3]`	`" Argentina"` → trim

CSS SELECTOR — DEVTOOLS "COPY SELECTOR" (EQUIVALENT)

FIELD	CSS SELECTOR TEMPLATE (ELEMENT DOM PATH)	OUTPUT @ I = 1
Year	`table.sortable.plainrowheaders.sticky-header > tbody > tr:nth-child({{index+1}}) > th a`	`"1930"`
Winner	`table.sortable.plainrowheaders.sticky-header > tbody > tr:nth-child({{index+1}}) > td:nth-child(2)`	`"Uruguay "` → trim
Score	`table.sortable.plainrowheaders.sticky-header > tbody > tr:nth-child({{index+1}}) > td:nth-child(3)`	`"4–2"`
Runners-up	`table.sortable.plainrowheaders.sticky-header > tbody > tr:nth-child({{index+1}}) > td:nth-child(4)`	`" Argentina"` → trim

Three differences between the two selector forms, worth knowing when you wire this up. ① **Column index:** CSS `:nth-child()` counts *all* siblings, and the Year cell is a `<th>`, so Winner/Score/Runners-up are `td:nth-child(2/3/4)` — whereas XPath's `td[…]` counts only `<td>` and so is `td[1/2/3]`. ② **Row arithmetic:** XPath evaluates `tr[{{index}}+1]` inside the predicate; CSS `:nth-child()` does *not* accept arithmetic, so the literal row number `index+1` is computed when the selector string is assembled (shown as `{{index+1}}`). ③ **Don't trust raw "Copy selector":** the live cells carry Parsoid-generated ids (e.g. `#mwng`) that change on every load, so DevTools' Copy selector would anchor to an unstable id. The class-anchored `:nth-child` chain above was verified to be stable and unique (one table matches).

Why these differ from the resource video — three verified changes since May 2024. ① **Prefix:** the video copies `//*[@id="mw-content-text"]/div[1]/table[4]/…`; Wikipedia now uses section-wrapped markup, so the same click yields `div[2]/section[2]/table[3]` (the old prefix resolves to nothing — re-copy at build time; the workflow keeps it in one variable). ② **Row offset:** in the video the sorter moves the header row into `<thead>`, making data row i = `tr[i]`; on today's page the header stays as `tbody` row 1, so data row i = `tr[i+1]`. ③ **Anchor nesting:** the video's `td[1]/a` works for 1930 but fails on rows where the link is span-wrapped (`td[1]/span/a`, e.g. 1954) — cell-level paths are the only tails uniform across all 10 rows.

F Expected result — the sheet after the run

Real values from the live table (not placeholders). The first 10 rows span 1930–1974: Wikipedia lists no rows for 1942/1946 (tournaments cancelled, WWII), so a fixed loop bound of 10 is exact.

	A	B	C	D
1	**Year**	**Winner**	**Score**	**Runners-up**
2	1930	Uruguay	4–2	Argentina
3	1934	Italy	2–1 (a.e.t.)	Czechoslovakia
4	1938	Italy	4–2	Hungary
5	1950	Uruguay	2–1 [n 3]	Brazil
6	1954	West Germany	3–2	Hungary
7	1958	Brazil	5–2	Sweden
8	1962	Brazil	3–1	Czechoslovakia
9	1966	England	4–2 (a.e.t.)	West Germany
10	1970	Brazil	4–1	Italy
11	1974	West Germany	2–1	Netherlands

Row 5's score keeps Wikipedia's footnote marker **[n 3]** (the 1950 "final" was the deciding match of a round-robin group) — see edge case #3 for the trade-off and the cleaner alternative.

Execution trace — the run, iteration by iteration

What each loop pass extracts and sends. Every value below was produced by evaluating the panel-E XPaths in a live browser session.

I	SOURCE ROW	REQUEST BODY `values` (STEP 2.7)	200 → `updatedRange`
1	tbody/tr[2]	`[[1930, "Uruguay", "4–2", "Argentina"]]`	`Sheet1!A2:D2`
2	tbody/tr[3]	`[[1934, "Italy", "2–1 (a.e.t.)", "Czechoslovakia"]]`	`Sheet1!A3:D3`
3	tbody/tr[4]	`[[1938, "Italy", "4–2", "Hungary"]]`	`Sheet1!A4:D4`
4	tbody/tr[5]	`[[1950, "Uruguay", "2–1[n 3]", "Brazil"]]`	`Sheet1!A5:D5`
5	tbody/tr[6]	`[[1954, "West Germany", "3–2", "Hungary"]]`	`Sheet1!A6:D6`
6	tbody/tr[7]	`[[1958, "Brazil", "5–2", "Sweden"]]`	`Sheet1!A7:D7`
7	tbody/tr[8]	`[[1962, "Brazil", "3–1", "Czechoslovakia"]]`	`Sheet1!A8:D8`
8	tbody/tr[9]	`[[1966, "England", "4–2 (a.e.t.)", "West Germany"]]`	`Sheet1!A9:D9`
9	tbody/tr[10]	`[[1970, "Brazil", "4–1", "Italy"]]`	`Sheet1!A10:D10`
10	tbody/tr[11]	`[[1974, "West Germany", "2–1", "Netherlands"]]`	`Sheet1!A11:D11`

Detour evaluates **true** on all 10 iterations (every field non-empty), so all 10 branches run — 10 POSTs, 10 × `updatedRows: 1`, zero skips.

G Edge cases & design decisions

#1 "First 10 rows" ≠ "first 10 World Cups by year"

1942 and 1946 were cancelled and have no table rows, so rows 1–10 cleanly map to 1930–1974. The loop bound of 10 needs no gap-handling.

#2 Append per row, not one batch call

The library has no accumulator step to build a 10-row array across iterations, and per-row appends pair perfectly with the Loop's skip-on-error rule: one bad row costs one row, not the whole write. 10 sequential calls sit far under the 60 write-req/min quota.

#3 The 1950 footnote marker

Row 4's score cell embeds a superscript note → cell text is `2–1[n 3]`. Chosen: extract the whole cell (keeps "(a.e.t.)" on rows 2 & 8) and accept the marker. Alternative: point ExtractHTML at `…/td[2]/a` for a pure `2–1` — at the cost of dropping "(a.e.t.)".

#4 Scores contain an en dash

`4–2` uses U+2013, not a hyphen. With `valueInputOption=RAW` the string is stored verbatim; no risk of Sheets coercing a dash-separated pair into a date.

#5 XPaths drift; the method doesn't

Three things changed between the resource video (May 2024) and today — all caught live: the table prefix (`div[1]/table[4]` → `div[2]/section[2]/table[3]`), the header row's home (`<thead>` → first `tbody` row, hence `tr[{{index}}+1]`), and the country-link nesting (`td[1]/a` vs `td[1]/span/a` , hence cell-level paths). The prefix lives in one variable, so the next drift is a one-field fix.

#6 Never sort before extracting

The table is user-sortable; clicking a header re-orders `<tr>` elements and would silently remap every index. The workflow performs zero clicks — extraction happens on the default chronological order.

#7 Why Parse number earns its place

Parse number converts `"1930"` → `1930` . Its decisive value is the per-row validity gate (a non-numeric year ⇒ error ⇒ loop skips that index); the numeric column A is a secondary nicety, subject to the library's type ambiguity in **#11**.

#8 Re-runs are not idempotent

`:append` always adds rows; running the workflow twice writes 20 rows. Clear `Sheet1!A2:D11` before a re-run (or point `RANGE` at a scratch tab while testing).

#9 Flag icons pad the country cells

Winner/Runners-up cells carry a stray non-breaking space next to the flag icon (`"Uruguay "` , `" Argentina"` — measured live). Text extraction normally trims whitespace; if a platform ever kept it, the Detour still passes (non-empty) and a one-time `TRIM()` in Sheets cleans column B/D.

#10 Why `tr[{{index}}+1]` is safe to template

After substitution the predicate is plain XPath arithmetic — `tr[1+1]` evaluates to `tr[2]` (confirmed in a live browser). Two equally valid alternatives, noted for completeness: give the **Loop a list** `[2…11]` so `index` is the row number directly and the `+1` disappears; or use the self-documenting selector `tr[position()>1][{{index}}]` ("skip the header, take the i-th data row"). The count-based `+1` form is chosen because it matches the *only* loop behaviour the library documents (index = 1…N).

#11 Parse number's ambiguous output type

The library row contradicts itself: the description says "converts … into its **Int** data type," but the Output column reads "**String**." The workflow doesn't hinge on the resolution — the step earns its place as the row-validity gate (a non-numeric year errors and the loop skips that index). If the output is genuinely a string, year still appends correctly; column A's numeric-vs-text nicety is the only thing at stake, and a one-time `=VALUE()` /format fix in Sheets settles it.

#12 Write authorization — why OAuth, not an API key

`values.append` is a write, so an API key alone is rejected (403). The chart uses Postman's OAuth 2.0 Authorization Code flow (panel D); a Google **service account** with the sheet shared to its address is the equivalent server-to-server alternative. Either yields the `Bearer` token the Call API step needs.

Verification. Both selector forms — XPath and CSS — were evaluated in a live Chrome session against every one of the 10 target rows (**80/80 checks** returned the expected text) on 7–8 Jul 2026, after cross-checking the served page HTML and Tennr's resource video; API shapes match the official `spreadsheets.values.append` reference.

Bhunesh Bansal
Implementation Engineer take-home · Tennr